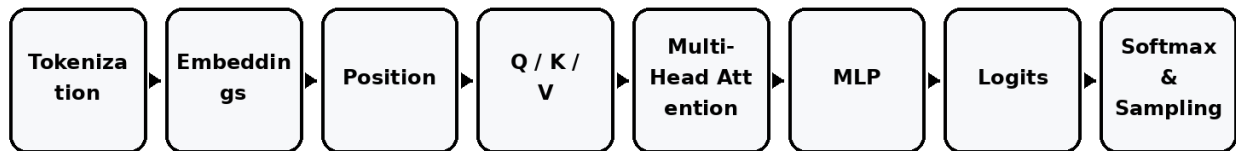


Inside an LLM Transformer

Tokenization → Embeddings → Position → Q/K/V → Multi-Head Attention → MLP → Logits → Softmax → Sampling

Inside a Decoder-Style Transformer



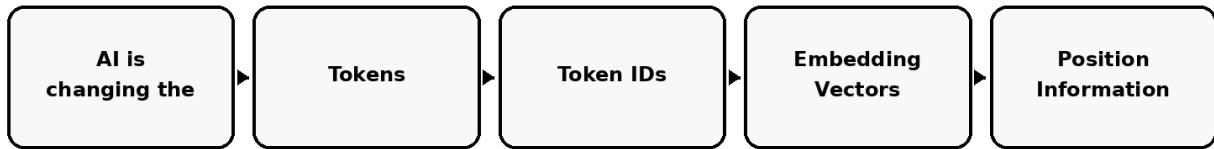
1. Example Prompt

We will trace the prompt “AI is changing the” through a decoder-style Transformer. The goal is to understand what happens inside the neural network before the model generates the next token.

2. Tokenization

The tokenizer converts text into tokens and then token IDs. A token may be a complete word, part of a word, punctuation, or another frequently occurring sequence. Exact tokenization and IDs depend on the model.

Text to Numerical Representations



3. Embeddings

Token IDs are indices. An embedding table maps each ID to a learned high-dimensional vector. These vectors are the numerical representations used by the Transformer.

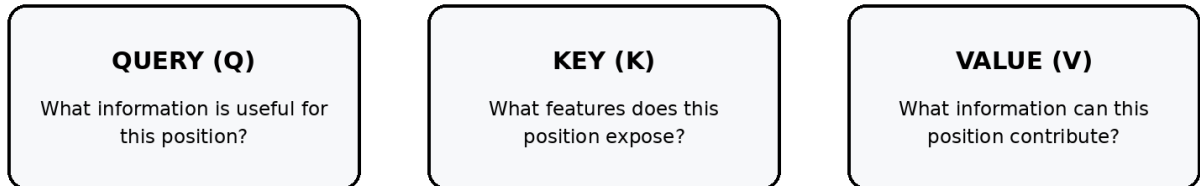
4. Positional Information

Attention alone does not inherently encode sequence order. Transformer architectures therefore incorporate positional information. Implementations vary; examples include learned positional embeddings and rotary positional techniques such as RoPE.

5. Q, K and V

Self-Attention: Query, Key and Value

$$Q = XW_Q \quad K = XW_K \quad V = XW_V$$



$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

Causal masking prevents a token from attending to future positions.

For each representation X , learned projection matrices create Query, Key, and Value vectors: $Q = XW_Q$, $K = XW_K$, and $V = XW_V$. The common intuition is that a Query represents what a position is looking for, a Key exposes features used for matching, and a Value contains information that can be mixed into the result. These descriptions are teaching analogies; mathematically, all three are learned vector projections.

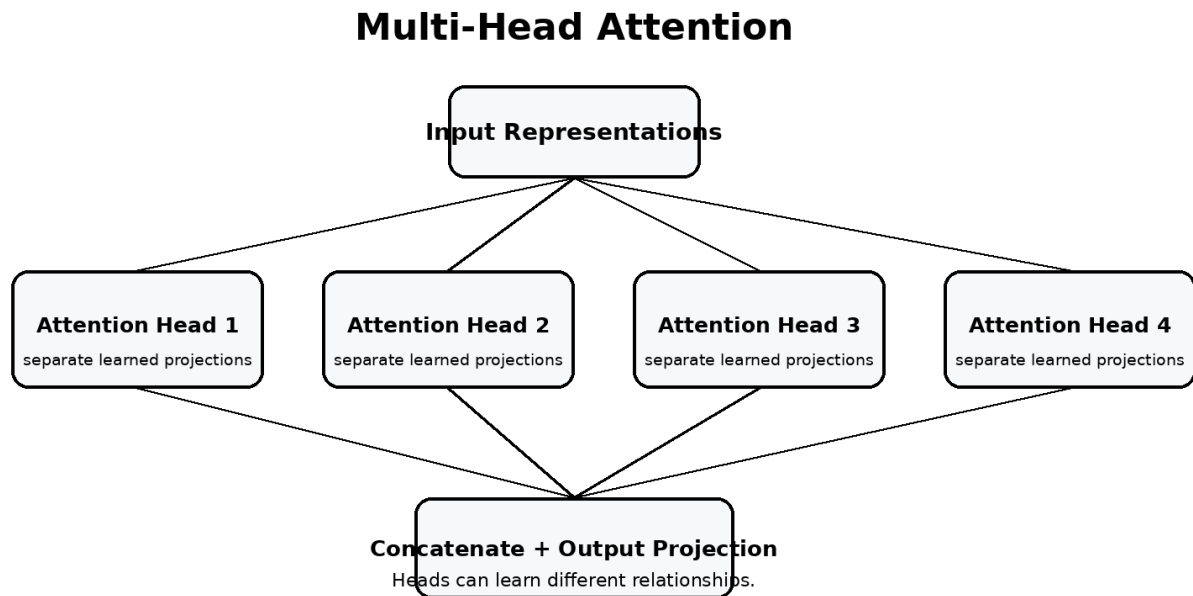
Scaled dot-product attention is commonly written as:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

6. Attention Scores and Causal Masking

QK^T produces compatibility scores between positions. The scores are scaled by $\sqrt{d_k}$ and passed through softmax to obtain normalized attention weights. In an autoregressive decoder, a causal mask prevents each position from attending to future tokens. Thus the token at position t can use information only from positions at or before t .

7. Multi-Head Attention



Instead of performing only one attention operation, the model uses multiple heads with separate learned projections. Their outputs are concatenated and passed through an output projection. Different heads may learn different useful relationships, although individual heads do not necessarily correspond to neat human-interpretable categories.

8. Residual Connections and Normalization

Transformer blocks use residual connections so transformed information is combined with an earlier representation rather than simply replacing it. Normalization is also used to stabilize computation. The exact placement of normalization differs across architectures, so diagrams often show either pre-normalization or post-normalization variants.

9. MLP / Feed-Forward Network

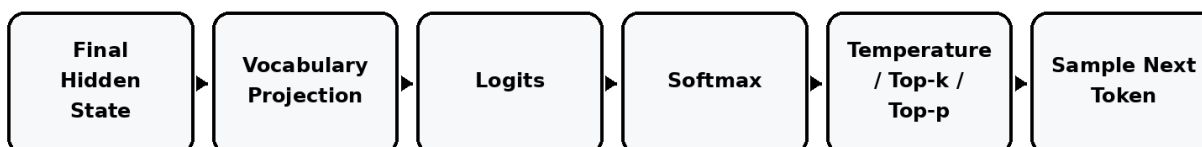
After attention, each position is processed by a feed-forward network or MLP. A simplified form is $\text{MLP}(x) = W_2 \sigma(W_1 x + b_1) + b_2$. Modern LLMs often use gated variants. A useful mental model is that attention mixes information across token positions, while the MLP performs nonlinear transformation within each position's representation.

10. Repeating Transformer Blocks

The attention, residual, normalization, and MLP operations are repeated across many Transformer blocks. The resulting hidden representations become progressively contextualized for predicting the next token.

11. Final Hidden State → Logits

From Hidden State to Next Token



For next-token generation, the final hidden representation is projected into vocabulary space. This produces one score, called a logit, for each vocabulary token. Higher logits generally indicate stronger preference before normalization.

Token	Illustrative Logit	Illustrative Probability
world	8.9	41%
way	7.8	19%
industry	7.5	15%
future	7.1	11%
banana	-1.2	very low

The values above are illustrative and are not actual outputs from a specific model.

12. Softmax

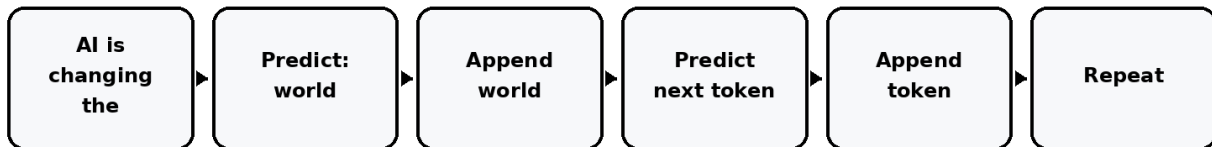
Softmax converts a set of logits z into a normalized probability distribution: $P_i = \exp(z_i) / \sum_j \exp(z_j)$. The probabilities can then be used by a decoding strategy to choose the next token.

13. Sampling and Decoding

Greedy decoding chooses the highest-scoring token. Sampling can introduce controlled variability. Temperature changes the sharpness of the distribution; top-k restricts candidates to the k most likely tokens; top-p (nucleus sampling) restricts candidates to a set whose cumulative probability reaches a chosen threshold. Production systems may combine or modify these techniques.

14. Autoregressive Generation

Autoregressive Generation



If the model chooses “world”, that token is appended to the context: “AI is changing the world”. The model then predicts another token. Formally, generation models $P(\text{token}_t \mid \text{token}_1, \dots, \text{token}_{t-1})$. This repeats until a stopping condition is reached.

Inference engines commonly optimize repeated generation using KV caching so previously computed key/value information does not need to be recomputed from scratch for every generated token.

15. Complete Internal Flow

Text prompt → tokenizer → token IDs

Token IDs → embeddings + positional information

Representations → Q/K/V projections

Q/K/V → causal multi-head self-attention

Attention → residual/normalization → MLP → residual/normalization

Repeat across Transformer blocks

Final hidden state → vocabulary projection → logits

Logits → softmax/decoding → selected next token

Append selected token → repeat autoregressively

16. Three Concepts to Remember

1. Embeddings turn discrete tokens into numerical vectors.
2. Attention creates context-dependent representations by selectively combining information across token positions.
3. The final objective during generation is to produce a distribution over the next token, select one according to a decoding strategy, append it, and repeat.

17. Bridge to AI Agents

Once this LLM inference loop is understood, the next architectural layer is easier to see. An AI agent wraps the model in a control loop that can choose actions, call tools, observe results, maintain state or memory, and decide what to do next. A simplified agent loop is: LLM → decision/action → tool execution → observation → updated context/state → next LLM decision → final answer.